The Olympus Process

01 The flow

The other tab covers what a submission is made of. This one covers what you run and in what order. Each step unlocks the next, and the steps get more expensive as you go. **Never spend on a later step while an earlier one is failing.**

01 **Prechecks.** Fast checks on each piece of your submission. Everything else is locked until these pass.

02 **Scope Gate.** A go or no-go on the task itself, before you spend on the expensive steps.

03 **The build.** Your Dockerfile becomes the image that every later step runs inside.

04 **Quality checks.** Your tests and solution actually run, plus deeper reviews of the task, tests, solution, and description.

05 **Agent rollouts.** Agents take real shots at your challenge to measure difficulty and fairness.

06 **The FP check.** Confirms the passing runs genuinely solved the task.

07 **Auto Review.** A full review of the whole submission, before a human sees it.

08 **Submit**, then review, and (hopefully) approval.

The **“Submission criteria”** panel on your submission tracks all of this live. When everything there is green, you can submit.

02 Prechecks

Prechecks are the cheap, fast layer. They validate each piece of your submission on its own before anything expensive runs:

- **Repo**: the URL and commit resolve, and the repo meets the requirements from the creation tab (stars, activity, language, license).
- **Problem and tests**: the description is well formed (length, formatting, no leftover URLs, matches its category), it isn't a near-duplicate of an existing problem, and the test patch is a valid diff with a working `test.sh`, no solution code, and no quest leaks.
- **Dockerfile**: follows the Dockerfile rules from the creation tab (base image, `/app` workdir, dependencies installed at build time, no tests in the build).
- **Solution patch**: a valid, cleanly structured diff.

Each precheck reports exactly what it found. Fix everything in one pass, then rerun; nothing downstream runs until they all pass and stay current. A warning isn't a failure: only failures block, but read each warning and confirm it's fine before moving on.

03 The Scope Gate

After prechecks pass, the Scope Gate reviews your task and gives a verdict: pass or drop. The quality checks later judge how good your submission is; the Scope Gate decides whether the task should exist at all, before you spend on the expensive steps. It digs into three things:

- **Duplicates on the platform**: it takes the similar submissions the similarity check surfaced and compares them in depth, the actual tasks and what their solutions touch, not just the wording, to confirm whether yours genuinely stands on its own or reworks an existing one.
- **Already handled upstream**: it investigates the repo's real history, PRs, issues, and discussions to check the capability wasn't already shipped, isn't sitting in an open PR, wasn't declined by the maintainers, and wasn't once removed on purpose.
- **Repo fit**: whether the task belongs in this repo at all. A task that contradicts the repo's stated goals or philosophy is a drop, no matter how well it's written.

These are the take-care rules from the creation tab, verified for real, so do that homework before spending here. A drop means the idea needs rethinking, not rewording. A pass unlocks the rest of the flow.

04 The build

Everything from here on runs inside a Docker image built from your Dockerfile. One image, used everywhere: the checks that execute your tests, and every agent rollout.

Use the Build Image button before running quality checks. Two things to keep in mind:

- The container runs with `--network none`, so anything your tests or the repo needs must be installed at build time.
- If you change the Dockerfile, rebuild. Checks and runs always use an image that matches your current content.

05 Quality checks

This is where your submission actually gets exercised. Run them from the checks panel once the build is green. What each one does and what passing means:

- **Verify Tests**: applies your test patch to the clean repo (no solution) and runs both modes. Passing means `base` passed and `new` failed, exactly the contract from the creation tab. If `new` passes here, your tests don't actually capture the missing behavior.
- **Verify Solution**: applies your solution on top and runs both modes again. Passing means everything passes: your solution implements the task without breaking existing tests.
- **Verify Flakiness**: runs the suites multiple times, with and without the solution, and fails if any test changes its result between identical runs. A flaky suite can't grade agents fairly, so this blocks everything downstream until it's fixed.
- **Test Quality**: reviews each of your hidden tests for fairness. Every test must check something the description states or a developer would discover in the repo. It also gives advisory feedback on coverage.
- **Task Quality**: grades the description and tests as a pair against our quality bar and lists exactly which criteria fail.
- **Solution Quality**: reviews your solution patch against the repo for completeness and code quality, and lists concrete issues to fix.
- **Description Quality**: reviews the description for tone, redundancy, and details an agent could find in the repo on its own, and suggests rewrites.

Read each output fully. These must pass and stay current before you can run agents: they're the most expensive step, and there's no point spending on them while a check is failing.

> **In the tutorial?** You're seeing a smaller version of this. The tutorial runs fewer checks and skips several of these steps entirely to make your first pass faster and easier. Everything here applies once you're creating real submissions.

06 Agent rollouts

A rollout is one agent taking a real shot at your challenge: it gets your problem description, your repo at the pinned commit, and your Docker environment, writes a solution, and gets graded against your tests. Rollouts are how you see whether your challenge actually challenges agents, and they feed the submission criteria directly.

A minimum number of finished rollouts is required before you can submit. The exact count sits in the “Submission criteria” panel on your submission form (it changes with product, so I won't hardcode it here). Runs only count once they finish, so don't leave them for the last minute.

Quick checks vs full batches

- A **quick check** is a single cheap run. Use it while you iterate: it tells you early whether agents understand the task and whether your environment holds up, without betting the wallet.
- A **full batch** fires several runs in one go. Once you feel confident in your challenge, use it to rack up finished runs and see if you clear the bar.
- The batch is also configurable from the same dialog, so you can set exactly which agents run and how many.

The banner above your runs reads the results as they land and tells you the next step: run a quick check, run the full batch, too easy, looks broken, or ready. Trust it over guessing.

How long they take

A rollout usually finishes within ~15 minutes. A big repo or a genuinely hard task can push that up to 90 minutes. Kick off your runs, go do something else, and come back for the results; watching the progress bar doesn't make the agents type faster.

> **Read the results before you edit.** Rollouts are pinned to the content they ran on. Editing any field after a rollout finishes marks it stale, it stops counting, and those tokens are gone. Same story for triggering more rollouts while a batch is still in flight: if those results force an edit, every rollout goes stale at once. One batch at a time, results first, edits second.

And read the fails: an agent failing because the task is hard is exactly what you want. An agent failing because a sentence was ambiguous is a revision waiting to happen. If agents are passing too often instead, the task is too easy for the difficulty bar; make it harder before spending the remaining runs.

07 The FP check

FP stands for false positive: an agent run that passed your tests without genuinely solving the task. It happens when tests are permissive enough to let a wrong or partial solution through, and it poisons your pass rate: a fake pass looks identical to a real one.

The FP check reviews every passing run and confirms each pass is legitimate. It powers the **“No false positives”** criterion: a single pass judged false fails it.

Two things to get right:

- **Run it after your run set is settled.** It covers the passing runs that exist when you trigger it, and edits stale it like everything else. Running it while you're still iterating is a waste.
- **If it finds a false positive, the fix is your tests.** The agent found a gap your tests don't cover. Strengthen them so the wrong solution can't pass, then rerun. And check whether the description needs an edit too: the strengthened tests must not assert something undiscoverable.

08 Auto Review

The Auto Review is a full review of your submission: the task, the tests, the solution, and your agent runs, covering the same ground a human reviewer will. Running it to completion is part of the submission criteria.

Its verdict won't block submitting, but it's the best preview you'll get of how review goes: anything it flags, a reviewer will likely flag too, and it's free to fix now versus a revision cycle later.

Run it last, once everything else is green. It reviews your submission as a whole, so running it while checks are failing or runs are missing tells you nothing useful, and any edit stales it. And it might grant you a **queue skip** if everything is solid: your submission goes straight to the final approval step with our managers instead of waiting in the review queue.

> You can also run it earlier, before or during your agent runs, as a helper. Just note that check reruns, content edits, and agent-run changes stale it, so you'll pay for a fresh run at the end.

09 After you submit

Submitting locks your submission for editing and sends it to review. From there, one of three things happens:

- **Approved**: it moves through a final quality pass, then finalizes. A finalized approval is what counts: payout, bonus tokens, and your approval stats.
- **Revision requested**: it comes back to you, editable, with the reviewer's feedback attached. Fix what's flagged, rerun whatever went stale, and resubmit. Revisions are normal; almost nobody lands everything on the first pass.
- **Rejected**: the reviewer found something disqualifying (most often a duplicate PR or a task that can't be made fair). If you believe the call is wrong, you can appeal with your reasoning.

Your review outcome is mostly decided before you submit: read your runs, fix what the checks and the Auto Review surface, and submit something you've already reviewed yourself.

10 Tokens and staleness

Running checks and rollouts consumes tokens:

- New contributors start with an initial token balance.
- Finalized approvals earn **bonus tokens**.
- Tokens replenish hourly based on your **contributor tier**, determined by your approval rate and number of finalized approvals.
- Costs generally follow the flow: prechecks are the cheapest, agent runs the most expensive; another reason to follow the order.
- Running checks on every small tweak will drain your balance fast.

Staleness

Every result is tied to the exact content it ran on. Edit any submission field after a check, rollout, or review has completed and that result goes **stale**: it no longer counts, and it must be rerun before you can submit, which costs tokens again.

That's what keeps results trustworthy: a green check always describes your current submission. Batch your edits: fix everything in one pass, then rerun once.

ON THIS PAGE

01 The flow
02 Prechecks
03 Scope Gate
04 The build
05 Quality checks
06 Rollouts
07 FP check
08 Auto Review
09 After submit
10 Tokens